

ICMP Header-Only File Transfer: Flow Control and Any-Size Files

Why the current approach breaks on larger files

Your current approach works for small files because the sender can finish before the receiver falls behind. For larger files, host A can transmit packets much faster than host B can receive, parse, store, and reassemble them. When that happens, the receiver's kernel/socket buffer can fill up, packets get dropped, and host B waits for packets that will never arrive. This is not mainly a file-size problem; it is a flow-control problem.

Flow control fix: stop-and-wait per chunk

The clean fix is to stop sending the whole file in one continuous blast. Instead, split the file into chunks and send only one chunk at a time. Host A sends all packets for chunk 0, then stops. Host B stores those packets by sequence number and only sends an ACK when the whole chunk has been received. Only after host A receives that ACK does it send chunk 1. This prevents host A from skipping ahead and prevents host B from being overwhelmed by too many packets at once.

How a chunk should be handled

Each chunk should have a fixed maximum number of packets, such as 512 or 1024. For example, with a 2-byte sequence and 2-byte data layout capped at 1024 packets, each chunk carries 2048 bytes of useful file data. The receiver should not save a chunk just because it sees the last sequence number. It should store every packet in a map or list by sequence number, then check that all expected sequence numbers for that chunk are present. For the final chunk, the receiver should use the original file size or final chunk size to know how many packets/bytes are actually expected.

Timeout and resend behavior

If host A sends a chunk and does not receive the ACK within a timeout, it should resend the same chunk instead of moving forward. This keeps the protocol simple while making it much more reliable. You do not need full TCP-style reliability for a practice tool; ACK-per-chunk with timeout/resend is enough to stop most failures caused by buffer overflow or occasional packet loss.

Moving from fixed-size transfers to any-size files

Right now, your design is limited because some metadata, such as file size, is stored in a single 16-bit field. That caps the value at 65535. To support files of any size, send transfer metadata before the data stream. The START phase should communicate the filename, full file size, chunk size, total chunk count, and packet layout mode. These metadata values can be sent across multiple header-only packets the same way you send normal bytes. Once both sides know the full file size and total chunks, the sender can transfer a 5 KB file, a 5 MB PDF, or a larger file using the exact same chunk loop.

Recommended overall flow

The complete flow should be: START packet(s), then chunk 0 data packets, ACK chunk 0, chunk 1 data packets, ACK chunk 1, and so on until the final chunk. After the final chunk is acknowledged, send an END packet. Host B then trims any padding from the final chunk using the original file size and writes the

reconstructed bytes to disk. This turns your current fixed small transfer into a repeatable file-transfer protocol that can scale to larger files without blasting the receiver.

Scapy vs raw sockets

Keep Scapy for low-volume control traffic such as port knocking, connect/disconnect, transfer-file request, and run-program request packets. Use raw sockets for high-volume transfer paths: file chunks, chunk ACKs, command byte streams, program output streams, and program output ACKs. That gives you a clean split: Scapy for the control plane, raw sockets for the data plane.